The operating system interface

SENTINEL: one operations centre, built in four labs.
Today: the overview, then Lab 1.

Machines watch.
People decide.

Four sensors send reports to one operations centre. A watch officer acts on what the display shows.

Watch 2026-282 has one open item: a vessel in the Juan de Fuca Strait with an identity on no filed list.

Your software gives the watch officer a true account, in time to act on it.

What you build: SENTINEL

Four sensors send reports. SENTINEL collects them, records each action, sorts them by importance and keeps one correct picture. Each lab adds one layer.

Lab 1 · Oct. 9

libsyscall

The interface to the operating system: the feed, the mission log and precise timestamps.

R6

Read a feed and lose none of it. Put a real date on each report. Record each report, in order.

Lab 2 · Oct. 30

libproc

One process for each sensor, and the channels that bring their reports to the centre.

R1 R2

Continue when one sensor stops. Let an urgent report overtake routine traffic.

Lab 3 · Nov. 20

libipc

Worker threads that share one processor and keep the track picture.

R3

Take thousands of reports each second. Decide that two reports are one object.

Lab 4 · Dec. 4

libsync

Protection for each thing that the workers share.

R4 R5

Keep the picture exact while many workers change it. Never freeze.

The marks of the four labs

Lab	Deliverable	Second part	Form	Total
1	5%	3%	Oral exam	**8%**
2	5%	3%	Oral exam	**8%**
3	5%	3%	Oral exam	**8%**
4	4%	4%	Video	**8%**
All four labs				**32%**

Each lab also has a prelab. It is due before the session of the lab.

The brief of each lab tells how the marker divides the marks.

A failed test does not stop the hand-in. You get marks for the checks that pass.

Timeline

Oct. 9	Oct. 22	Oct. 30	Nov. 12	Nov. 20	Dec. 2	Dec. 4	Dec. 11
Lab 1 session	Lab 1 due	Lab 2 session Lab 1 oral exam	Lab 2 due	Lab 3 session Lab 2 oral exam	Lab 3 due	Lab 4 session Lab 3 oral exam	Lab 4 due Lab 4 video

Each deliverable is due by midnight, on Brightspace. Each oral exam is in the session of the next lab.

How to submit

1. Make the files that the brief of the lab lists.
2. Run `make submit`. It builds from the source, runs the tests and writes one zip.
3. Upload the zip to Brightspace before the deadline.
4. Oral exam: six to ten minutes at your machine, in the next session. Run your lab, then answer questions about your code.

Where to find the details

`README.md`	The list of all documents
`docs/manual/`	SENTINEL Lab Manual: facts for all labs
`docs/setup/`	Setup Guide: your machine and repository
`docs/submit/`	Submission Guide: marks, oral exams, hand-in
`docs/lab1/`	Lab 1 Brief, Guide and prelab
`syscall.h`	Each function of Lab 1, and its errors

4%

Lab 4: a video in place of the oral exam

Show the whole system, from end to end.

Explain the project, and the problems of shared data that you solve in Lab 4.

Due Dec. 11. Upload it with the Lab 4 zip. The Lab 4 brief gives the length and format.

01

Lab 1: libsyscall

Read the feed, give the time and write the mission log. Requirement R6.

Lab 1 at a glance

Oct. 7

Prelab due by midnight.
Necessary, no marks.

Oct. 9

Lab session.

Oct. 22

Deliverable due by midnight.
80 marks.

Oct. 30

Oral exam in the Lab 2 session.
20 marks.

The 80 marks of the deliverable

16	16	12	12	12	12
Acceptance test	Course tests	Your tests	Style and errors	Makefile, valgrind	Three files agree

What you write in Lab 1

`src/io.c, clock.c, log.c, proc.c`	The 15 functions of syscall.h
`Makefile`	Library, demo, test and clean
`tests/test_*.c`	Four tests: io, clock, log, proc
`tests/demo_lab1.c`	The demonstration program

The data flow

contactfeed
Given

feed.txt

demo_lab1
Your libsyscall

missions.log

make watch
localhost:8080

How Lab 1 works

1

You get a feed

Sensor reports arrive as lines of text in a file.

`feed.txt`

→

2

Read the stream

System calls read the lines. Continue until all bytes are read.

`read()`

→

3

Add the time

Each line gets a timestamp from the clock.

`clock_gettime()`

→

4

Write the log

Each line goes to the mission log at once. No line is lost.

`missions.log`

→

5

Look at the process

/proc shows the state of the program while it runs.

`/proc/<pid>`

Your library does steps 2 to 5. The demo runs them on a real feed.

The demo

```
build/demo_lab1 <feed-file> [mission-log] [--follow IDLE_MS]
```

It reads the feed to the end. It writes one mission log line for each feed line.

With --follow, it waits for new lines. It stops when no line arrives for IDLE_MS.

It prints a summary, the process report and a leak check: (balanced).

Done when, then hand in

- [ ] make builds your library and demo with no warning.
- [ ] make test passes.
- [ ] make accept1 passes all 34 checks.
- [ ] make lab1 PROFILE=dirty shows rejected above zero.
- [ ] The demo prints (balanced).
- [ ] With --follow, the demo stops on IDLE_MS, not on EAGAIN.
- [ ] valgrind reports no leak and no error.

Then hand in

```
make submit
sentinel-lab1-<name>.zip
```

Put the three files of one run in docs/lab1/ first.

make submit builds from the source, runs the tests and writes the zip.

Upload the zip to Brightspace by Oct. 22, midnight.

Questions?

If you have no questions, start work on your lab.

`docs/lab1/lab1_brief.pdf`